

To download YouTube videos reliably, the industry standard is **yt-dlp**. Older Python libraries like pytube or pafy are heavily broken because YouTube constantly updates its platform to block scrapers. yt-dlp is actively maintained by developers to bypass these restrictions, auto-merge HD video with audio tracks, and prevent sudden speed throttling.

Prerequisites

Before running the code, open your terminal or command prompt and install the package:
pip install yt-dlp

Note on High Definition (1080p or higher): YouTube splits HD video streams and audio streams into separate files. If you want yt-dlp to stitch them together into a high-quality .mp4 or .mkv automatically, make sure you have **FFmpeg** installed on your computer. If FFmpeg isn't installed, the script will still work but will default to downloading the highest pre-merged quality (usually 720p).

The Python Downloader Script

```
import os
from yt_dlp import YoutubeDL

def progress_hook(d):
    """
    A custom callback function that catches the download status
    and prints a readable progress bar in the terminal.
    """
    if d['status'] == 'downloading':
        # Calculate progress percentage safely
        total_bytes = d.get('total_bytes') or d.get('total_bytes_estimate') or 0
        downloaded_bytes = d.get('downloaded_bytes', 0)

        if total_bytes > 0:
            percentage = (downloaded_bytes / total_bytes) * 100
            # Get current download speed (formatted nicely)
            speed = d.get('_speed_str', 'N/A')
            # Extract ETA (Estimated Time of Arrival)
            eta = d.get('_eta_str', 'N/A')

            # Print a single rewriting line using carriage return (\r)
            print(f"\rDownloading... {percentage:.1f}% | Speed: {speed} | ETA: {eta}", end="",
                  flush=True)

        elif d['status'] == 'finished':
            print("\nDownload finished! Processing/merging tracks...")

def download_youtube_video(video_url, output_folder="Downloads"):
    """
    Downloads a video from YouTube using optimized options via yt-dlp.
    """
```

```

"""
# Create the output directory if it doesn't exist
if not os.path.exists(output_folder):
    os.makedirs(output_folder)

# Configuration options for yt-dlp
ydl_opts = {
    # 'bestvideo+bestaudio/best' tells it to download the absolute best video track
    # and best audio track, then merge them together.
    'format': 'bestvideo+bestaudio/best',

    # Set the directory and name format for the output file
    # %(title)s gets replaced with the video name, %(ext)s with the file extension
    'outtmpl': os.path.join(output_folder, '%(title)s.%(ext)s'),

    # Restrict filenames to safer characters (avoids weird OS errors with emojis/symbols)
    'restrictfilenames': True,

    # Connect our status tracking function up to the downloader loop
    'progress_hooks': [progress_hook],

    # Mimic a standard browser header to reduce the chance of getting a 403 Forbidden/bot
    error
    'http_headers': {
        'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,
like Gecko) Chrome/120.0.0.0 Safari/537.36',
    }
}

print(f"Analyzing video link: {video_url}")

try:
    # Initialize the YoutubeDL context engine with our custom options dictionary
    with YoutubeDL(ydl_opts) as ydl:
        # Download the video directly
        ydl.download([video_url])
        print(f"\n Success! Check your '{output_folder}' directory.")

except Exception as e:
    print(f"\n An error occurred while trying to download: {e}")

if __name__ == "__main__":
    print("=" * 50)
    print("      Optimized YouTube Video Downloader      ")
    print("=" * 50)

    # Take input from the user
    url_input = input("Paste the YouTube video URL here: ").strip()

```

```
if url_input:
    download_youtube_video(url_input)
else:
    print("Error: URL cannot be blank.")
```

Step-by-Step Implementation

1. **Install Dependency**

Terminal Setup

Install the core library via your terminal layout using pip:

2. **Save File**

Script Composition

Save the code block directly above onto your computer as downloader.py.

3. **Run & Download**

Execution Phase

Run the script inside your terminal execution tree: Paste any valid YouTube watch link when prompted, then press Enter. Your file will drop directly inside a new folder named Downloads/ relative to where your script sits.